

re Module Functions

PATTERN / SYNTAX	DESCRIPTION & EXAMPLE
<code>re.search(pat, s)</code>	Scan string for first match; returns Match object or None EXAMPLE: <code>re.search(r'\d+', 'id: 42')</code> → Match
<code>re.match(pat, s)</code>	Match characters only at the start of string; returns Match or None EXAMPLE: <code>re.match(r'\w+', 'foo bar')</code> → 'foo'
<code>re.fullmatch(pat, s)</code>	Match the entire string from start to end; returns Match or None EXAMPLE: <code>re.fullmatch(r'\d{4}', '2026')</code> → Match
<code>re.findall(pat, s)</code>	Return all non-overlapping matches as a list of strings or tuples EXAMPLE: <code>re.findall(r'\d+', '10, 20, 30')</code> → ['10', '20', '30']
<code>re.finditer(pat, s)</code>	Return an iterator yielding Match objects over all matches EXAMPLE: <code>[m.start() for m in re.finditer(r'\d+', s)]</code>
<code>re.sub(pat, repl, s)</code>	Replace occurrences of pattern with repl (string or function) EXAMPLE: <code>re.sub(r'\s+', '-', 'hello world')</code> → 'hello-world'
<code>re.subn(pat, repl, s)</code>	Same as <code>re.sub()</code> , returns tuple (new_string, number_of_subs) EXAMPLE: <code>re.subn(r'\d', '#', '1a2')</code> → ('#a#', 2)
<code>re.split(pat, s)</code>	Split string by occurrences of pattern (optional maxsplit) EXAMPLE: <code>re.split(r'[;,]\s*', 'a, b; c')</code> → ['a', 'b', 'c']
<code>re.compile(pat, flags)</code>	Compile pattern into a reusable Pattern object for performance EXAMPLE: <code>pattern = re.compile(r'\b\w+\b', re.I)</code>
<code>re.escape(pattern)</code>	Escape all special characters in string for literal matching EXAMPLE: <code>re.escape('https://')</code> → 'https://'
<code>re.purge()</code>	Clear the internal regular expression memory cache EXAMPLE: <code>re.purge()</code>

Match Object Methods

PATTERN / SYNTAX	DESCRIPTION & EXAMPLE
<code>m.group(0)</code>	Return the entire matched substring (same as <code>m.group()</code>) EXAMPLE: <code>m.group()</code> → '2026-08-18'
<code>m.group(n)</code>	Return subgroup matched by index number (1-based) EXAMPLE: <code>m.group(1)</code> → '2026'
<code>m.group('name')</code>	Return subgroup matched by named capture (?P<name>...) EXAMPLE: <code>m.group('year')</code> → '2026'
<code>m.groups()</code>	Return a tuple containing all captured subgroups EXAMPLE: <code>m.groups()</code> → ('2026', '08', '18')
<code>m.groupdict()</code>	Return a dict containing all named subgroups {name: value} EXAMPLE: <code>m.groupdict()</code> → {'year': '2026'}
<code>m.start(group)</code>	Return start character index of match (defaults to 0) EXAMPLE: <code>m.start()</code> → 4
<code>m.end(group)</code>	Return end character index of match (defaults to 0) EXAMPLE: <code>m.end()</code> → 8
<code>m.span(group)</code>	Return tuple of (start, end) index positions (defaults to group 0) EXAMPLE: <code>m.span()</code> → (4, 8)
<code>m.expand(template)</code>	Do backslash substitution on template string (\1 or \g<name>) EXAMPLE: <code>m.expand(r'Year: \g<year>')</code>
<code>m.string</code>	The original string passed to <code>match()</code> or <code>search()</code> EXAMPLE: <code>m.string</code> → source text

Python Regex Flags

PATTERN / SYNTAX	DESCRIPTION & EXAMPLE
<code>re.IGNORECASE</code>	Perform case-insensitive matching (short alias: <code>re.I</code>) EXAMPLE: <code>re.findall(r'cat', 'CAT Cat', re.IGNORECASE)</code>
<code>re.MULTILINE</code>	<code>^</code> and <code>\$</code> match start/end of each line in addition to string (short alias: <code>re.M</code>) EXAMPLE: <code>re.findall(r'^\w+', text, re.MULTILINE)</code>
<code>re.DOTALL</code>	<code>.</code> matches any character including newline <code>\n</code> (short alias: <code>re.S</code>) EXAMPLE: <code>re.search(r'<p>.*?</p>', html, re.DOTALL)</code>
<code>re.VERBOSE</code>	Ignore whitespace & enable inline comments (<code>#</code>) in pattern (short alias: <code>re.X</code>) EXAMPLE: <code>re.compile(r'\d{3} # code', re.VERBOSE)</code>
<code>re.ASCII</code>	Make <code>\w</code> , <code>\d</code> , <code>\s</code> , <code>\b</code> match ASCII only (not full Unicode) (short alias: <code>re.A</code>) EXAMPLE: <code>re.findall(r'\w+', text, re.ASCII)</code>
<code>re.UNICODE</code>	Enable Unicode matching (default in Python 3) (short alias: <code>re.U</code>) EXAMPLE: <code>re.findall(r'\w+', 'résumé', re.UNICODE)</code>
<code>re.LOCALE</code>	Make <code>\w</code> , <code>\b</code> locale-dependent for bytes patterns only (short alias: <code>re.L</code>) EXAMPLE: <code>re.compile(rb'\w+', re.LOCALE)</code>
<code>re.IGNORECASE re.MULTILINE</code>	Combine multiple flags using bitwise OR operator (<code> </code>) EXAMPLE: <code>re.compile(r'^foo', re.IGNORECASE re.MULTILINE)</code>

Python-Specific Syntax & Groups

PATTERN / SYNTAX	DESCRIPTION & EXAMPLE
<code>r"..."</code>	Raw string literal — prevents Python escape sequence collisions EXAMPLE: <code>r'\bword\b'</code> vs <code>'\\bword\\b'</code>
<code>(?P<name>...)</code>	Named capturing group — accessible by name in match object EXAMPLE: <code>(?P<year>\d{4})</code> → <code>m.group('year')</code>
<code>(?P=name)</code>	Backreference to named group within the same pattern EXAMPLE: <code>(?P<q>['"])*?(?P=q)</code> → <code>"quoted"</code>
<code>\g<name></code>	Backreference to named group in <code>re.sub()</code> replacement string EXAMPLE: <code>re.sub(r'(?P<v>\d+)', r'\g<v>px', s)</code>
<code>\g<1></code>	Backreference to numbered group in <code>re.sub()</code> (prevents ambiguity) EXAMPLE: <code>re.sub(r'(\d+)', r'\g<1>0', s)</code>
<code>(?#comment)</code>	Inline regex comment — ignored by the engine EXAMPLE: <code>\d{3}(?#area code)-\d{4}</code>
<code>(?(id)yes no)</code>	Conditional: matches 'yes' if group id matched, else 'no' EXAMPLE: <code>(\d{3}(?(1)\d))- \d{4}</code>
<code>\A</code>	Match only at start of string (ignores <code>re.MULTILINE</code>) EXAMPLE: <code>re.search(r'\Astart', text)</code>
<code>\Z</code>	Match only at end of string (ignores <code>re.MULTILINE</code>) EXAMPLE: <code>re.search(r'End\Z', text)</code>
<code>\b</code>	Word boundary (raw string <code>r'\b'</code> prevents ASCII backspace) EXAMPLE: <code>r'\bcat\b'</code> → <code>'cat'</code> not <code>'category'</code>